南開大學

信息隐藏技术课程实验报告

实验三：图像信号的常用处理方法

学　　院　密码与网络空间安全学院

专　　业　信息安全、法学双学位班

学　　号　2313815

姓　　名　段俊宇

班　　级　信息安全、法学双学位班

一、实验要求

1、 学习智慧课堂：图像信号处理基础。

2、 FFT。

3、 DWT。

4、 DCT。

忽略上述三种变换中的高频成分直至低频成分，分别去除系数的 15%、30%、50%、80%，并做逆变换，然后用 PSNR 进行客观评价，观察他们的影响。

二、实验原理

二维离散傅里叶变换：该算法能将信号或序列从时域（或空间域）转换到频域，从而展示出信号中包含的不同频率成分及其强度。涉及的公式如下所示：

正变换：

$$F(u,v)=\frac{1}{MN}\sum_{x=0}^{M-1}\sum_{y=0}^{N-1}f(x,y)e^{-j2\pi\left(\frac{ux}{M}+\frac{vy}{N}\right)}$$

反变换：

$$f(x,y)=\sum_{u=0}^{M-1}\sum_{v=0}^{N-1}F(u,v)e^{j2\pi\left(\frac{ux}{M}+\frac{vy}{N}\right)}$$

$$x=0,1,\dots,M-1;\quad y=0,1,\dots,N-1$$

幅度谱：

$$|F(u,v)|=[R^2(u,v)+I^2(u,v)]^{\frac{1}{2}}$$

相位谱：

$$\phi(u,v)=arctan\frac{I(u,v)}{R(u,v)}$$

离散傅里叶变换有如下特性：

线性性：

$$a_1f_1(x,y)+a_2f_2(x,y)\ \leftrightarrow\ a_1F_1(u,v)+a_2F_2(u,v)$$

比例性：

$$f(ax,by)\leftrightarrow\frac{1}{|ab|}F\left(\frac{u}{a},\frac{v}{b}\right)$$

空间位移：

$$f(x-x_0,y-y_0) \leftrightarrow F(u,v)e^{\frac{-j2\pi(ux_0+vy_0)}{N}}$$

频率位移：

$$f(x,y)e^{\frac{j2\pi(u_0x+v_0y)}{N}} \leftrightarrow F(u-u_0,v-v_0)$$

旋转不变性：图像的空间域 f(x,y)和频率域 F(u,v)可以分别用极坐标表示 f(r,q)和 F(w,f)

$$f(r,\theta+\theta_0) \leftrightarrow F(\omega,\phi+\theta_0)$$

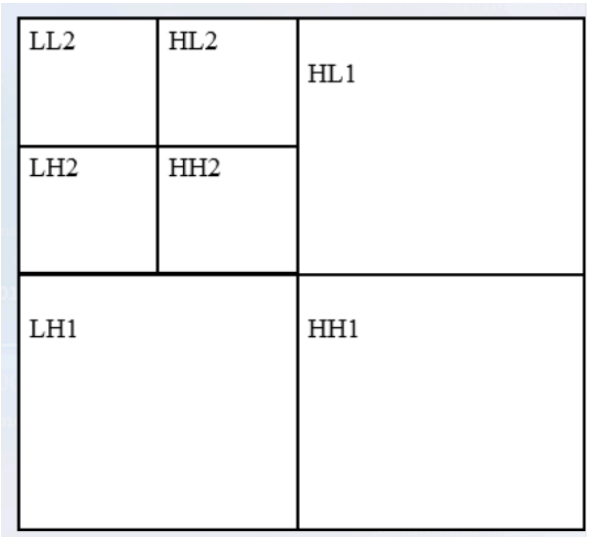
平均值（直流分量）：

$$F(0,0)=\frac{1}{N^2}\sum_{x=0}^{N-1}\sum_{y=0}^{N-1}f(x,y)$$

二维离散小波变换（DWT）：该算法用于信号处理中的时频分析，它通过对基本小波函数进行离散化的伸缩和平移，将信号分解成一系列不同尺度和位置的小波系数。小波变换在低频部分具有较高的频率分辨率，较低的时间分辨率；在高频部分具有较高的时间分辨率，较低的频率分辨率，符合低频信号变化缓慢而高频信号变化迅速的特点。其主要公式为：

$$W(a,b)=\frac{1}{\sqrt{a}}\int x(t)\psi^*\left(\frac{t-b}{a}\right)dt$$

其中，$\psi(t)$为母小波，a 为尺度因子，b 为平移因子。通过离散化 a 和 b，可实现对信号的高效分解与重构。一级分解包括近似部分(LL)、水平方向细节部分(HL)、垂直方向细节部分(LH)、对角线方向细节部分(HH)，二级分解各部分如下图所示：



二维离散余弦变换（DCT）：该算法是一种将信号从空间域或时域转换到

频域的变换方法，其核心思想是使用不同频率的余弦函数作为基函数。

正变换：

$$T(u,v)=\sum_{x=0}^{N-1}\sum_{y=0}^{N-1}f(x,y)g(x,y,u,v)$$

$$g(x,y,0,0)=\frac{1}{N}$$

$$g(x,y,u,v)=\frac{2}{N}cos\frac{(2x+1)u\pi}{2N}cos\frac{(2y+1)v\pi}{2N}$$

反变换：

$$f(x,y)=\sum_{u=0}^{N-1}\sum_{v=0}^{N-1}T(u,v)h(x,y,u,v)$$

系数：

$$C(0,0)=\frac{1}{N}\sum_{x=0}^{N-1}\sum_{y=0}^{N-1}f(x,y)$$

$$C(u,v)=\frac{2}{N}\sum_{x=0}^{N-1}\sum_{y=0}^{N-1}f(x,y)\,cos\frac{(2x+1)\pi u}{2N}cos\frac{(2y+1)\pi v}{2N}$$

重构：

$$f(x,y)=\frac{1}{N}C(0,0)+\frac{2}{N}\sum_{u=0}^{N-1}\sum_{v=0}^{N-1}C(u,v)cos\frac{(2x+1)\pi u}{2N}cos\frac{(2y+1)\pi v}{2N}$$

三、实验环境

Matlab_R2025b，Image Processing Toolbox，Wavelet Toolbox

四、实验过程

1. FFT 变换处理

如下所示，我将使用注释的方式对代码进行解释。

```
% FFT 压缩函数
function [img_recon, psnr_val] = fft_compress(I, keep_ratio)
    % FFT 变换，将零频移到中心
    fa = fft2(I);
    ffa = fftshift(fa);
    % 获取频谱尺寸
    [M, N] = size(ffa);
    % 计算保留的像素数量
    total_pixels = M * N;
```

```
    keep_pixels = round(total_pixels * keep_ratio);
    % 创建圆形掩膜，保留低频
    [X, Y] = meshgrid(1:N, 1:M);
    center_x = (N+1)/2;
    center_y = (M+1)/2;
    radius = sqrt((X - center_x).^2 + (Y - center_y).^2);
    radius_sorted = sort(radius(:));
    threshold_radius = radius_sorted(keep_pixels);
    mask = radius <= threshold_radius;
    % 应用掩膜，丢弃高频
    ffa_compressed = ffa .* mask;
    % 逆变换
    fa_recon = ifftshift(ffa_compressed);
    img_recon = real(ifft2(fa_recon));
    % 归一化
    img_min = min(img_recon(:));
    img_max = max(img_recon(:));
    if img_max > img_min
        img_gray = (img_recon - img_min) / (img_max - img_min);
    else
        img_gray = img_recon;
    end
    % 计算 PSNR
    psnr_val = psnr_calc(I, img_gray);
    % 二值化
    img_recon = img_gray > 0.5;
end
```

这部分代码首先进行 FFT 变换并将零频移到中心，接着按照比例对低频和高频部分进行处理，最后归一化并计算 PSNR，为重建图像和 PSNR 对比做准备。

2. DWT 变换处理函数

如下所示，我将使用注释的方式对代码进行解释。

```
% DWT 压缩函数
function [img_recon, psnr_val] = dwt_compress(I, keep_ratio)
    % 一级小波
    [ca1, ch1, cv1, cd1] = dwt2(I, 'db4');
    % 二级小波
    [ca2, ch2, cv2, cd2] = dwt2(ca1, 'db4');
    % 所有系数拼接
    coeffs = [ca2(:); ch2(:); cv2(:); cd2(:); ch1(:); cv1(:); cd1(:)];
    total_coeffs = length(coeffs);
```

```
    keep_coeffs = round(total_coeffs * keep_ratio);
    % 各子带长度
    len_ca2 = numel(ca2);
    len_ch2 = numel(ch2);
    len_cv2 = numel(cv2);
    len_cd2 = numel(cd2);
    len_ch1 = numel(ch1);
    len_cv1 = numel(cv1);
    len_cd1 = numel(cd1);
    % 初始化
    coeffs_compressed = zeros(size(coeffs));
    % 优先保留最低频 LL2
    ca2_keep = min(len_ca2, keep_coeffs);
    coeffs_compressed(1:ca2_keep) = ca2(1:ca2_keep);
    remaining = keep_coeffs - ca2_keep;
    % 高频按能量排序
    if remaining > 0
        high_coeffs = [ch2(:); cv2(:); cd2(:); ch1(:); cv1(:); cd1(:)];
        [~, idx_high] = sort(abs(high_coeffs), 'descend');
        high_keep = min(length(high_coeffs), remaining);
        coeffs_compressed(len_ca2 + idx_high(1:high_keep)) = ...
            high_coeffs(idx_high(1:high_keep));
    end
    % 重新拆分系数
    p = 0;
    ca2_compressed = reshape(coeffs_compressed(p+1:p+len_ca2), size(ca2));
    p = p + len_ca2;
    ch2_compressed = reshape(coeffs_compressed(p+1:p+len_ch2), size(ch2));
    p = p + len_ch2;
    cv2_compressed = reshape(coeffs_compressed(p+1:p+len_cv2), size(cv2));
    p = p + len_cv2;
    cd2_compressed = reshape(coeffs_compressed(p+1:p+len_cd2), size(cd2));
    p = p + len_cd2;
    ch1_compressed = reshape(coeffs_compressed(p+1:p+len_ch1), size(ch1));
    p = p + len_ch1;
    cv1_compressed = reshape(coeffs_compressed(p+1:p+len_cv1), size(cv1));
    p = p + len_cv1;
    cd1_compressed = reshape(coeffs_compressed(p+1:p+len_cd1), size(cd1));
    % 先重建一级 LL
    ca1_recon = idwt2(ca2_compressed, ch2_compressed, cv2_compressed, cd2_compressed, 'db4');
    ca1_recon = ca1_recon(1:size(ch1,1), 1:size(ch1,2));
    % 再重建原图
```

```
    img_recon = idwt2(ca1_recon, ch1_compressed, cv1_compressed, cd1_compressed, 'db4');
    % 裁剪尺寸
    img_recon = img_recon(1:size(I,1), 1:size(I,2));
    % 归一化
    img_min = min(img_recon(:));
    img_max = max(img_recon(:));
    if img_max > img_min
        img_gray = (img_recon - img_min) / (img_max - img_min);
    else
        img_gray = img_recon;
    end
    % 计算 PSNR
    psnr_val = psnr_calc(I, img_gray);
    % 二值化
    img_recon = img_gray > 0.5;
end
```

这部分代码对图像进行二级小波分解，通过保留最低频并将高频按照能量排序，重新拆分系数，再重建原图并实现归一化，最后计算 PSNR。

3. DCT 变换处理函数

如下所示，我将使用注释的方式对代码进行解释。

```
% DCT 压缩函数
function [img_recon, psnr_val] = dct_compress(I, keep_ratio)
    % DCT 变换
    c = dct2(I);
    % 获取 DCT 系数尺寸
    [M, N] = size(c);
    % 计算保留的系数数量
    total_coeffs = M * N;
    keep_coeffs = round(total_coeffs * keep_ratio);
    % 使用 Zigzag 扫描保留低频系数
    zigzag_idx = zigzag_order_full(M, N);
    coeffs_1d = c(zigzag_idx);
    coeffs_1d(keep_coeffs+1:end) = 0;
    c_compressed = zeros(size(c));
    c_compressed(zigzag_idx) = coeffs_1d;
    % 逆 DCT 变换
    img_recon = idct2(c_compressed);
    % 归一化
    img_min = min(img_recon(:));
    img_max = max(img_recon(:));
    if img_max > img_min
```

```
        img_gray = (img_recon - img_min) / (img_max - img_min);
    else
        img_gray = img_recon;
    end
    % 计算 PSNR
    psnr_val = psnr_calc(I, img_gray);
    % 二值化
    img_recon = img_gray > 0.5;
end
```

这部分代码使用 Zigzag 扫描保留低频系数，之后进行逆变换和归一化处理，并计算 PSNR 系数。Zigzag 的函数实现如下所示：

```
% Zigzag 扫描顺序函数
function idx = zigzag_order_full(M, N)
    % 生成 M x N 矩阵的 Zigzag 扫描索引
    idx = zeros(M*N, 1);
    count = 1;

    for sum_val = 0:(M+N-2)
        if mod(sum_val, 2) == 0
            % 从下往上扫描
            for i = min(sum_val, M-1):-1:max(0, sum_val-(N-1))
                j = sum_val - i;
                idx(count) = sub2ind([M, N], i+1, j+1);
                count = count + 1;
            end
        else
            % 从上往下扫描
            for j = min(sum_val, N-1):-1:max(0, sum_val-(M-1))
                i = sum_val - j;
                idx(count) = sub2ind([M, N], i+1, j+1);
                count = count + 1;
            end
        end
    end
end
```

4. PSNR 计算函数

如下所示，我将使用注释的方式对代码进行解释。

```
% PSNR 计算函数
function psnr_val = psnr_calc(img_orig, img_recon)
    % 计算峰值信噪比
    mse = mean((img_orig(:) - img_recon(:)).^2);
```

```
    if mse == 0
        psnr_val = 100;
    else
        psnr_val = 20 * log10(1.0 / sqrt(mse));
    end
end

```

5. 主函数

```
clear; clc; close all;
% 读入图像并预处理
b = imread('../Picture/1.png');
b = rgb2gray(b);
I = im2double(b);

figure(1);
imshow(b);
title('(a)原图像');

% 参数设置
discard_ratios = [15, 30, 50, 80];
keep_ratios = 1 - discard_ratios/100;

% 存储 PSNR 结果和重建图像
psnr_fft = zeros(1, length(keep_ratios));
psnr_dct = zeros(1, length(keep_ratios));
psnr_dwt = zeros(1, length(keep_ratios));

img_fft_all = cell(1, length(keep_ratios));
img_dct_all = cell(1, length(keep_ratios));
img_dwt_all = cell(1, length(keep_ratios));

% FFT 原始幅度谱
fa = fft2(I);
ffa = fftshift(fa);
figure(2);
imshow(log(abs(ffa)+1), []);
title('(b)FFT 原始幅度谱');

figure(3);
% 降采样减少数据点，显示更清晰
step = 4;  % 每 4 个点取 1 个
ffa_small = abs(ffa(1:step:end, 1:step:end));
surf(ffa_small, 'EdgeColor', 'none', 'FaceAlpha', 0.8);
```

```
view(45, 30);
colormap("parula");
colorbar;
title('(c)FFT 能量谱峰图');
xlabel('频率');
ylabel('频率');
zlabel('能量');
lighting phong;
light('Position', [1 1 1]);
grid on;

% FFT 变换及压缩
for i = 1:length(keep_ratios)
    keep = keep_ratios(i);
    discard = discard_ratios(i);

    % 调用 FFT 压缩函数
    [img_recon, psnr_val] = fft_compress(I, keep);

    % 存储结果
    img_fft_all{i} = img_recon;
    psnr_fft(i) = psnr_val;
end

% DCT 原始系数
c = dct2(I);
figure(4);
imshow(c);
title('(d)DCT 原始变换系数');

figure(5);
% 使用对数尺度处理 DCT 系数
log_c = log(abs(c) + 1);
surf(log_c, 'EdgeColor', 'none', 'FaceAlpha', 0.9);
view(45, 30);
colormap(jet);
colorbar;
title('(e)DCT 系数能量分布', 'FontSize', 12);
xlabel('水平频率', 'FontSize', 10);
ylabel('垂直频率', 'FontSize', 10);
zlabel('log(|DCT 系数|)', 'FontSize', 10);
shading interp;
lighting gouraud;
light('Position', [1 1 1]);
```

```
grid on;

% DCT 变换及压缩
for i = 1:length(keep_ratios)
    keep = keep_ratios(i);
    discard = discard_ratios(i);

    % 调用 DCT 压缩函数
    [img_recon, psnr_val] = dct_compress(I, keep);

    % 存储结果
    img_dct_all{i} = img_recon;
    psnr_dct(i) = psnr_val;
end

% DWT 原始系数
[ca1, ch1, cv1, cd1] = dwt2(I, 'db4');
[ca2, ch2, cv2, cd2] = dwt2(ca1, 'db4');
nbcol = size(ca2,1);
nbc = size(ch1,1);
cod_ca2 = wcodemat(ca2, nbcol);
cod_ch2 = wcodemat(ch2, nbcol);
cod_cv2 = wcodemat(cv2, nbcol);
cod_cd2 = wcodemat(cd2, nbcol);
cod_ch1 = wcodemat(ch1, nbc);
cod_cv1 = wcodemat(cv1, nbc);
cod_cd1 = wcodemat(cd1, nbc);
tt = [cod_ca2, cod_ch2; cod_cv2, cod_cd2];
tt = imresize(tt, size(ch1));
figure(6)
image([tt, cod_ch1; cod_cv1, cod_cd1])
colormap(gray(255))
title('(f)二级 DWT 变换系数')

% DWT 变换及压缩
for i = 1:length(keep_ratios)
    keep = keep_ratios(i);
    discard = discard_ratios(i);

    % 调用 DWT 压缩函数
    [img_recon, psnr_val] = dwt_compress(I, keep);

    % 存储结果
    img_dwt_all{i} = img_recon;
```

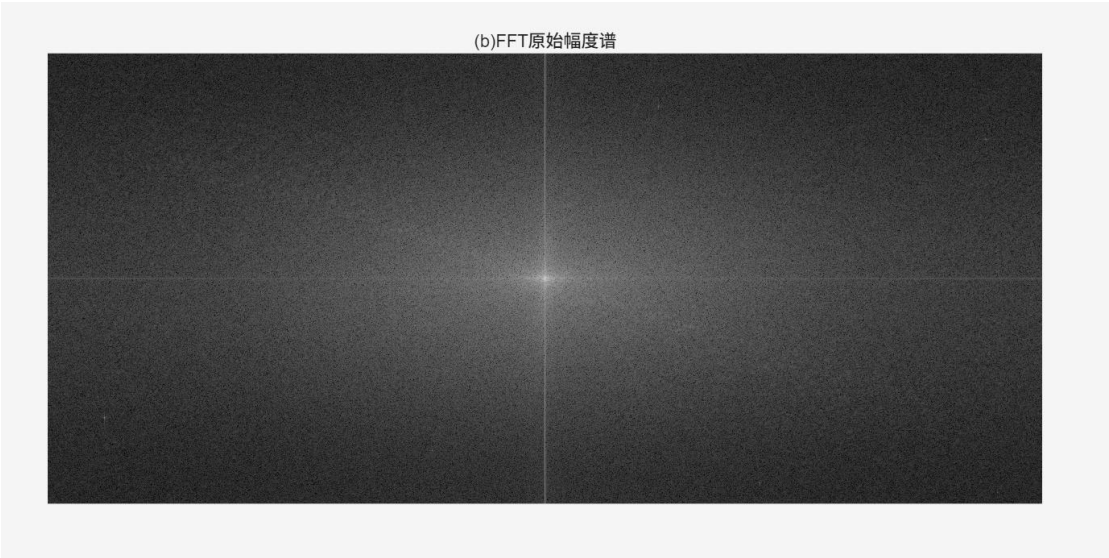
```
    psnr_dwt(i) = psnr_val;
end

% 显示 FFT 不同丢弃比例的重建图像
figure(7);
for i = 1:length(discard_ratios)
    subplot(1, 4, i);
    imshow(img_fft_all{i});
    title(sprintf('FFT 丢弃%d%%\nPSNR=%.2f dB', discard_ratios(i), psnr_fft(i)), 'FontSize', 10);
end
sgtitle('FFT 变换：不同丢弃比例下的重建图像对比', 'FontSize', 14);

% 显示 DCT 不同丢弃比例的重建图像
figure(8);
for i = 1:length(discard_ratios)
    subplot(1, 4, i);
    imshow(img_dct_all{i});
    title(sprintf('DCT 丢弃%d%%\nPSNR=%.2f dB', discard_ratios(i), psnr_dct(i)), 'FontSize', 10);
end
sgtitle('DCT 变换：不同丢弃比例下的重建图像对比', 'FontSize', 14);

% 显示 DWT 不同丢弃比例的重建图像
figure(9);
for i = 1:length(discard_ratios)
    subplot(1, 4, i);
    imshow(img_dwt_all{i});
    title(sprintf('DWT 丢弃%d%%\nPSNR=%.2f dB', discard_ratios(i), psnr_dwt(i)), 'FontSize', 10);
end
sgtitle('DWT 变换：不同丢弃比例下的重建图像对比', 'FontSize', 14);

% 显示 PSNR 对比曲线
figure(10);
plot(discard_ratios, psnr_fft, 'r-o', 'LineWidth', 2, 'MarkerSize', 8, 'MarkerFaceColor', 'r'); hold on;
plot(discard_ratios, psnr_dct, 'g-s', 'LineWidth', 2, 'MarkerSize', 8, 'MarkerFaceColor', 'g');
plot(discard_ratios, psnr_dwt, 'b-^', 'LineWidth', 2, 'MarkerSize', 8, 'MarkerFaceColor', 'b');
grid on;
xlabel('高频成分丢弃比例 (%)', 'FontSize', 12);
ylabel('PSNR (dB)', 'FontSize', 12);
```

```
title('三种变换压缩性能对比曲线', 'FontSize', 14);
legend('FFT', 'DCT', 'DWT', 'Location', 'southwest');
set(gca, 'FontSize', 11);
```

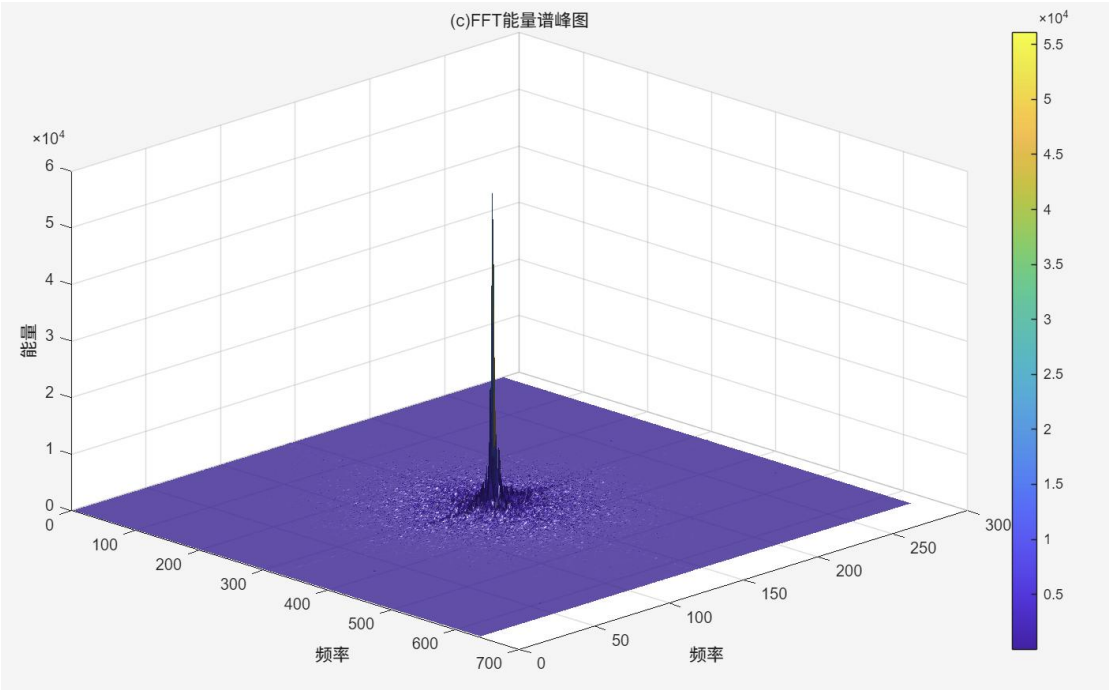
在本次实验中我选择了第一次实验中的图片作为样本，将其命名为 1.png。主函数中调用前面的三种处理函数对图像分别进行处理，最后展示了三种方法在丢弃不同比例的处理图像、PSNR 对比折线图等结果。

如下是原始图像的灰度图像：

如下所示是原始图像的 FFT 幅度谱和能量谱峰图：



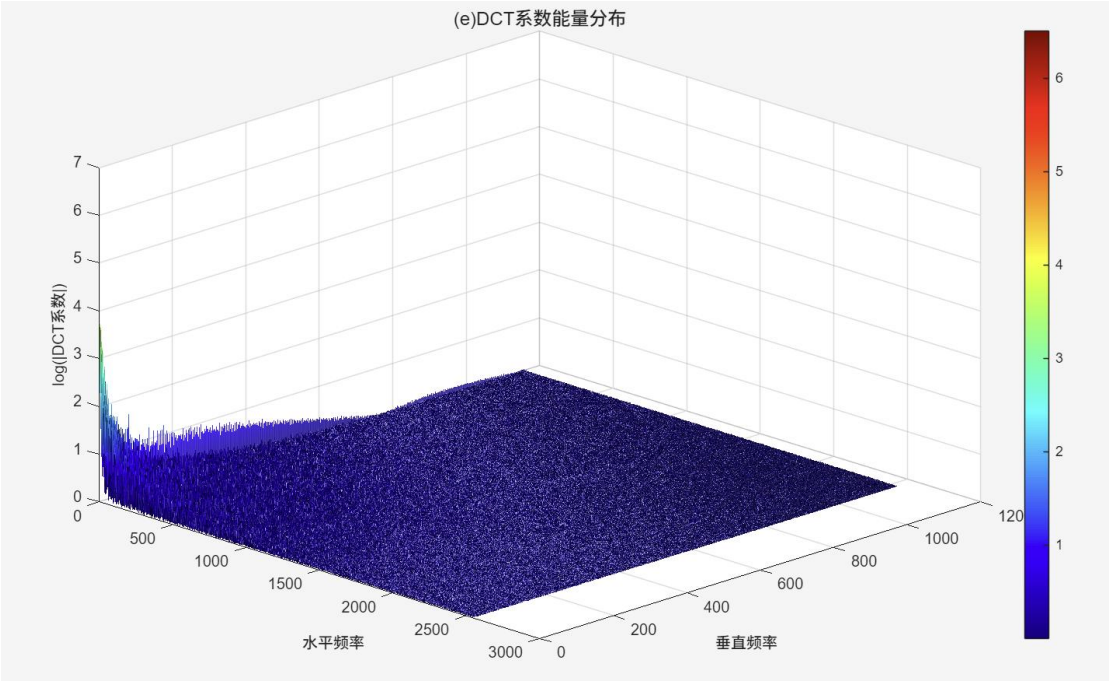
由于 FFT 中将零频移到了图像中心，因此幅度谱显示的中心最亮，然后向四周开始变暗，这说明能量主要集中在低频区域。



能量谱峰图的效果更加明显。由于零频移到了图像中心，因此中间的能量最高；大部分均为紫色，而中间颜色很深说明低频占大部分；放大图像后发现中间掺杂着浅色，而整体来看只能看到深色，说明高频在低频中间分布，占很小的比例。

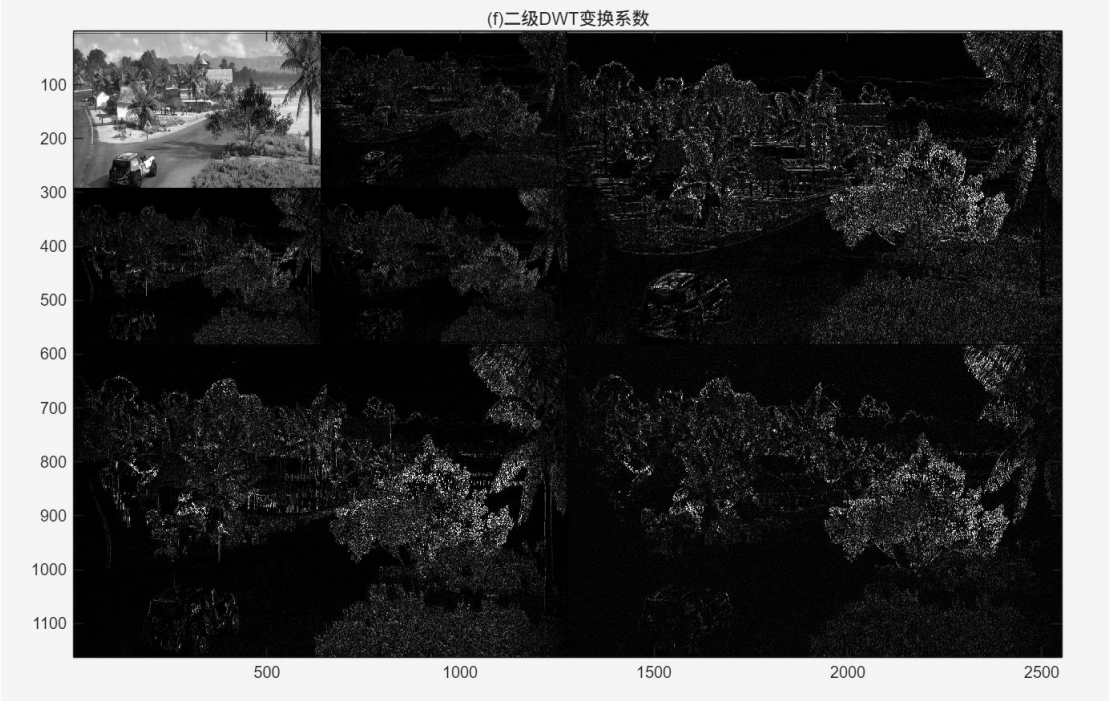
如下所示是原始图像的 DCT 变换系数和能量分布图：

从图中可以看出，能量集中在左上角，右下角能量趋近于零。



DCT 系数能量分布图展示的更加明显。由于能量中心在左上角，因此左上角高度最高；深色占大部分，其中掺杂着彩色，这说明低频占绝大多数，高频占少部分，符合图像的频率分布规律。

如下所示是 DWT 的变换系数：

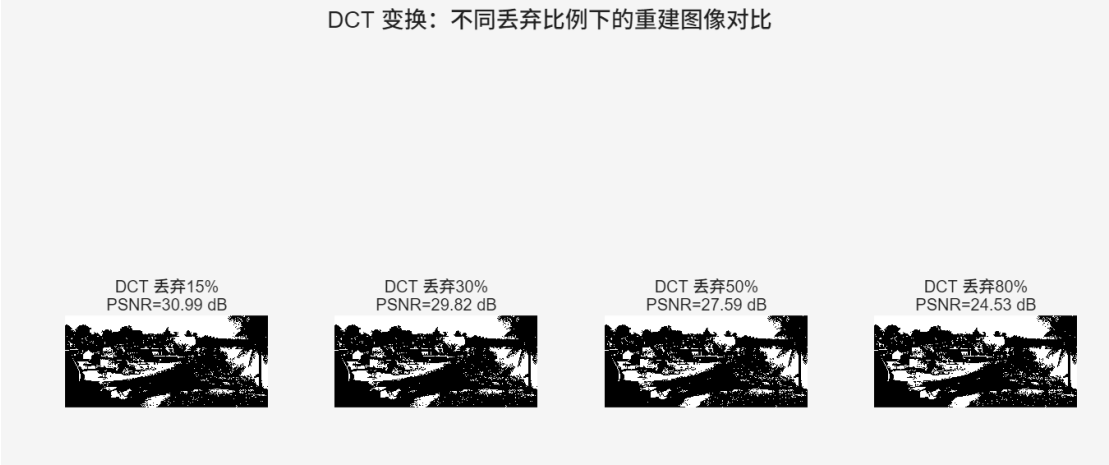


二级 DWT 分解更加清晰的展现了图像的结构，左上角的最低频包含了图像的主要结构，而二级高频包含了图像的中等细节，一级高频包含了图像的边缘和纹理细节。

如下所示是 FFT 丢弃不同比例的高频部分的处理结果：

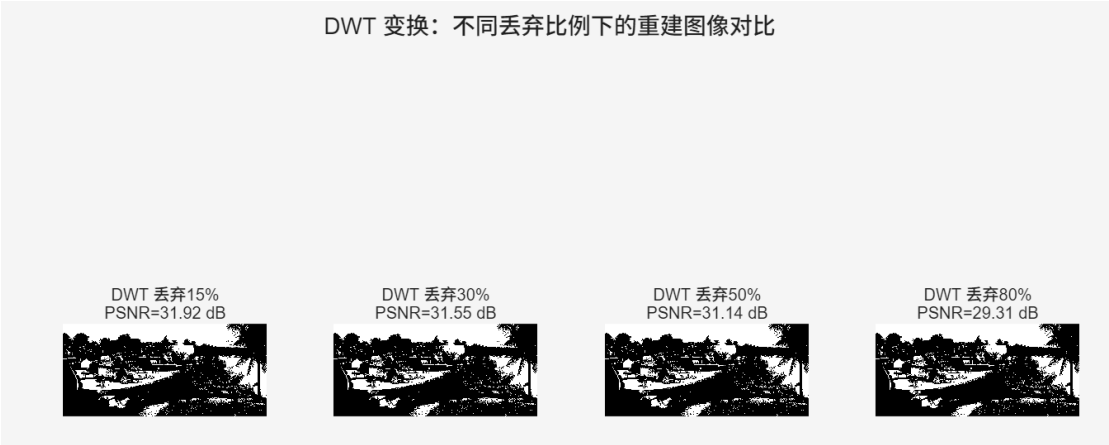
从图中可以看出，在丢弃不同比例的高频部分后，重建图像基本相同，只有边缘会随着丢弃比例增加逐渐模糊，但不是很明显。

如下所示是 DCT 丢弃不同比例的高频部分的处理结果：



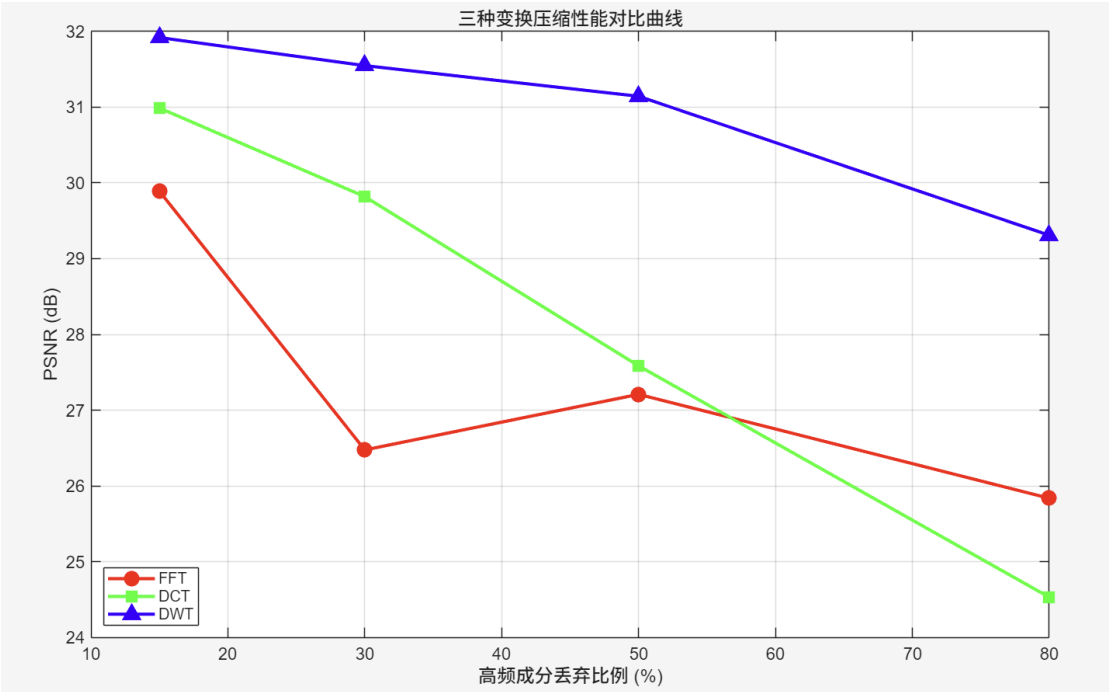
从图中可以看出，在丢弃不同比例的高频部分后，重建图像基本相同。而相对 FFT 的处理结果，相同丢弃比例下，边缘保持较好，细节损失较少。

如下所示是 DWT 丢弃不同比例的高频部分的处理结果：



从图中可以看出，在丢弃不同比例的高频部分后，重建图像基本相同。并且和前两种方法比对后，发现二级小波变换在高丢弃比例下仍能保持图像的主要结构。

下面是三种变换去除不同比例高频信号后的 PSNR 对比图：



从上面的图像可以看出，DCT 在低丢弃比例下性能较好，DWT 在高丢弃比例下表现稳定，FFT 在高丢弃比例下性能下降明显这说明 DWT 的边缘保持性能和稳定性都更好，

五、实验结论及心得体会

本次实验中我巩固了课上学习的图像信号的常用处理方法，在实验过程中使用 Matlab 调用工具库，实现了三种变换对原始图像的处理，并按不同

比例去除高频信号。通过比对 FFT 的幅度谱和能量谱峰图，验证了 FFT 将零频放到图像中心的特点；而 DCT 的变换系数和能量分布图展现了低频占绝大多数、高频在低频中掺杂的特点；DWT 的变换系数则展现了不同频率对图像结构的贡献。重建图像的 PSNR 对比展示出 DWT 在图像压缩中的优势，在高压缩率下，DWT 能够更好地保留图像的结构信息，具有较高的实用价值。